



AI & Software Engineering Workshop

Շաբաթ 1, Օր 3: Հրամաններ և հիշողություն — Windows

Edik Simonian, Ամառ 2026

Դեկորատորները 60 վայրկյանում

```
@bot.message_handler(commands=["start"], func=is_allowed)
def cmd_start(message):
    ...
```

- Դեկորատորը **գրանցում** է քո ֆունկցիան բոտի մոտ
- «Երբ հաղորդագրությունը համընկնում է `/start`-ին, կանչիր այս ֆունկցիան»
- Դու ինքդ `cmd_start` -ը չես կանչում, դա անում է բոտը
- Սա այն ամենն է, ինչ պետք է իմանաս ցանկացած հրաման ավելացնելու համար

Ներկառուցվածները՝ կարդա նախքան գրելը

Հրաման	Ինչ է անում
/start	Ողջույնի հաղորդագրություն (արդեն խմբագրել ես)
/help	Թվարկում է հրամանները (երեկ խմբագրել ես)
/reset	Մաքրում է քո զրույցի պատմությունը
/about	Մոդելի, պահոցի և հոստինգի մասին տեղեկություն

/reset -ը մաքրում է *պատմությունը*: Մի թոպե, ինչ պատմություն: **Դա այսօրվա երկրորդ կեսն է:**

Կենդանի կոդ՝ `/joke`

Կոշտ կոդով գրված կատակը միշտ նույնն է պատմում: Եկեք հրամանին ստիպենք **մտածել՝**

```
@bot.message_handler(commands=["joke"], func=is_allowed)
def cmd_joke(message):
    reply = ask_ai(message.from_user.id, "Tell one short, clean programming joke.")
    bot.send_message(message.chat.id, reply)
```

`ask_ai(user_id, prompt)` -ն ուղարկում է մեկանգամյա հարցում և վերադարձնում պատասխանը — **ամեն անգամ թարմ կատակ**, ոչ մի ցուցակ պահելու կարիք չկա:

1. Ավելացրու մշակիչը (handler) `bot/handlers.py` -ում
2. Ավելացրու `/joke` -ը `/help` -ին
3. Վերագործարկիր, թեստավորիր

Քո հերթն է՝ ընտրիր մեկը

Յուրաքանչյուրը հարցնում է AI-ին — ամեն անգամ թարմ արդյունք, ճիշտ ինչպես `/joke`-ը՝

- `/quote`՝ ինքնատիպ ոգևորող տող
- `/fact`՝ զարմանալի փաստ (թող AI-ն քեզ զարմացնի)
- `/compliment`՝ ինչ-որ մեկի օրը պայծառացրու
- `/roll`՝ զառի նետում — **բացառությունը**, զուտ Python (`random.randint`), առանց AI-ի

Նույն բաղադրատոմսը՝ մշակիչ → `/help`-ի գրառում → վերագործարկում → թեստ:

Առաջինը կառուցիր ձեռքով, հաջորդը զուգակցիր Claude Code-ի հետ, բայց վերանայման շրջանի ժամանակ դու ես այն բացատրում:

Մակարդակ վեր՝ քո բառերը փոխանցիր AI-ին

`/roast <name>`՝ կարդա հրամանից հետո եկած բառերը և ուղարկիր AI-ին՝

```
@bot.message_handler(commands=["roast"], func=is_allowed)
def cmd_roast(message):
    name = message.text.split(maxsplit=1)[1] if " " in message.text else "you"
    reply = ask_ai(message.from_user.id, f"Write a short, playful, friendly roast of {name}.")
    bot.send_message(message.chat.id, reply)
```

`message.text.split()`՝ ահա սա է արգումենտների վերլուծությունը:

(Repo-ի `/model` մշակիչը նույն հնարքն է անում, այն ակտիվացնում է Շաբաթ 2-ում:)

Ամրացրու թեստով

- Բացիր `tests/test_handlers.py` -ը՝ տես, թե ինչպես է `/start` -ը թեստավորվում
- Գրիր նույն տեսքի թեստը `pn` հրամանի համար
- `.\make.ps1 test`՝ կանաչ՝ լոկալ
- `git push`՝ կանաչ նաև GitHub Actions-ում

Առանց թեստի հրամանը հետագայում լուռ կկոտրվի:

Կոդի վերանայման շրջան

1. Տեղափոխվիր մեկ արժու ձախ՝ քո հարևանի էկրանին
2. Կարդա այդ էկրանի մշակիչը **բարձրաձայն**, տող առ տող
3. Բացատրիր, թե ինչ է անում յուրաքանչյուր տողը, հարցրու, եթե չգիտես
4. Մեկ հաճոյախոսություն, մեկ առաջարկ, ապա վերադարձիր

Ուրիշների կողմ կարդալը իրական ծրագրային ինժեներիայի կեսն է:

Խնդիրը

1. Ասա քո բոտին՝ «*իմ սիրած գույնը կապույտն է*»
2. Ապա հարցրու քո **հաջորդ իսկ հաղորդագրությունում՝** «*հྱրն է իմ սիրած գույնը*»
3. **Նա գաղափար անգամ չունի:**

Յուրաքանչյուր պատասխան մոռանում է նախորդը. սա Օր 1-ի «stateless mode» ծանուցումն է: Հրամանները պատրաստ են, **հիմա բոտին հիշողություն ենք տալիս:**

Բանալի-արժեք պահոց = պահարաններ

- Պահարանների մի պատ 
- **Բանալին** = պահարանի համարը → `chat:12345`
- **Արժեքը** = ինչ էլ որ ներսում դնես → զրույցը
- `get(key)` / `set(key, value)`՝ ահա ողջ API-ն
- Մերն ունի նաև **TTL**՝ պահարաններ, որոնք ինքնամաքրվում են 30 օր հետո

Redis, DynamoDB, memcached, նույն գաղափարը: Մերը SQLite է՝ տվյալների բազա մեկ ֆայլում:

Միացրու այն

`.env``

```
SQLITE_PATH=./bot.db
```

- Վերագործարկիր → ասա սիրած գույնդ → նորից վերագործարկիր → հարցրու
- **Այն հիշում է:** Հայտնվեց `bot.db` ֆայլը՝ ահա հիշողությունը
- Ոչ գրանցում, ոչ սերվեր, ոչ քարտ: Պարզապես ֆայլ է
- Զնչիր տողը → սահուն վերադարձ stateless ռեժիմին (փորձիր)

Ինչպես է բոտը հիշում զրույցները

Մոդելը **վիճակ չպահող (stateless)** է՝ ամեն API կանչ մոռանում է նախորդը: Հիշողությունը *մեր* կողմն է, ոչ թե մոդելը:

```
# bot/ai.py – runs on every message
history = get_history(user_id)           # load past turns from bot.db
history.append({"role": "user", "content": text})

messages = [{"role": "system", "content": SYSTEM_PROMPT}, *history]
reply = generate(user_id, messages)      # replay the whole chat

history.append({"role": "assistant", "content": reply})
save_history(user_id, history)           # write back: last 20, 30-day TTL
```

Բանալիավորված `chat:<your id>`-ով, ցուցակը պահվում է `bot.db`-ում, այնպես որ այն **գոյատևում է վերագործարկումներից հետո**:

Կենդանի կոդ՝ `/remember` և `/recall`

```
@bot.message_handler(commands=["remember"], func=is_allowed)
def cmd_remember(message):
    note = message.text.split(maxsplit=1)[1] if " " in message.text else ""
    store.set(f"note:{message.from_user.id}", note)
    bot.send_message(message.chat.id, "Saved!")
```

`/recall` -ը հայելային պատկերն է՝ `store.get(f"note:{...}"):`

Ինքնուրույն կառուցում՝ լիարժեք նշումների հնարավորություն

- `/remember <text>`՝ ավելացրու նշում (ոչ թե փոխարինիր)
- `/recall`՝ թվարկիր **քոլոր** պահված նշումները
- `/forget`՝ մաքրիր դրանք

Հուշում՝ պահոցը պահում է **տողեր (strings)**: Ցուցակ պահելու համար մուտքի ժամանակ `json.dumps` արա, ելքի ժամանակ՝ `json.loads` :

Այսօր → Վաղը

Այսօր քո բոտն ունի հրամաններ, որ **մտածում են**, և հիշողություն, որ **գոյատևում է վերագործարկումներից** — և դու պարզեցիր, թե իրականում ինչ է `/reset` -ը զրոյացնում:

Վաղը. տեղակայում (deploy): Բոտը դառնում է կենդանի՝ համացանցում և թարմացվում ամեն `git push` -ի դեպքում, այնպես որ այլևս քո նոութբուքի կարիքը չի ունենա: